ACADEMIC REPORT

Mobile Application Development

Assa Ticket: A Flutter-Based Mobile Bus Ticketing System for Chad

Lecturer: **Dr Sampah David**

Prepared by: Deubaybe Dounia

Date: April 25, 2026

Platform: Flutter (Android / iOS)

Figma Design: https://www.figma.com/design/leOkNoyVtznOo1LW2yOsnJ/Assa-ticket?node-id=0-1&t=aPzCQ3B3mg4h8LUT-1

Github: https://github.com/Dounia337/assa_ticket.git

Api separated: https://github.com/Dounia337/api.git

Youtube video: https://youtu.be/YTO8ztz1FMA?si=nCzNDus0x9iBH7h5

CHAPTER 1 — INTRODUCTION

1.1 Background and Problem Statement

Chad relies almost entirely on road transport for intercity travel. Despite this, the existing bus ticketing infrastructure is wholly manual: passengers must visit terminals in person, booking is done verbally or on paper, seat availability is unknown remotely, and no digital record of transactions exists. Payment in Chad is increasingly mobile-based, with Moov Money and Airtel Money widely adopted across the country, yet no ticketing platform currently integrates with these services.

1.2 Purpose and Scope

Assa Ticket is a Flutter-based mobile application designed to digitalise the intercity bus ticketing process in Chad. Its tagline — "Votre voyage à travers le Tchad commence ici" — reflects its ambition to serve as the single entry point for bus travel. The application covers the complete passenger journey: trip search, seat selection, passenger and luggage entry, mobile money payment, and digital ticket receipt. An integrated administrator module allows bus operators to manage routes, vehicles, trips, bookings, and promotions within the same application.

1.3 Target Users

The application serves two user groups. Travellers are French-speaking adults across Chad, including users with limited digital literacy, who need to book intercity journeys from their mobile phones. Bus company administrators are operational staff responsible for managing routes, vehicles, scheduling, booking confirmation, and customer communications.

CHAPTER 2 — REQUIREMENTS

2.1 Key Functional Requirements

The table below summarises the most significant functional requirements of the system.

Req. ID	Feature	Description
FR-01/02	Phone + OTP Authentication	Users register and log in using a Chadian phone number (+235). A six-digit OTP is verified before access is granted.
FR-04	Role-Based Access Control	The system supports two roles: USER and ADMIN. Administrative screens are accessible only to the designated admin account.
FR-09/11	Trip Search	Users search trips by specifying origin, destination, and date. Only trips matching the selected date are returned.
FR-13	GPS Nearby Trips	With location permission, the app detects the user's nearest city via the Haversine formula and shows upcoming departures.
FR-16	Visual Seat Selection	A seat map renders all seats per trip colour-coded as AVAILABLE, OCCUPIED, or BLOCKED. Users tap to select seats.
FR-18	Luggage Declaration	Users declare item count and weight. Surcharges apply: 0–15 kg free, 15–30 kg at 1,000 FCFA, above 30 kg at 2,500 FCFA.
FR-19/20	Mobile Money Payment	Users pay via Moov Money or Airtel Money and upload a payment screenshot for admin verification, or opt to pay

		at the station.
FR-21	Digital Ticket Generation	A unique ticket number (AS-XXXXX) and QR code are generated on booking confirmation and stored as a persistent digital record.
FR-37	Booking Confirmation/Rejection	Administrators review all pending bookings and confirm or reject them, triggering push notifications to the affected passenger.
FR-41	Statistics Dashboard	The admin dashboard displays aggregate figures: total bookings, confirmed, pending, rejected, total revenue, users, and active trips.

2.2 Key Non-Functional Requirements

The application targets a French-speaking, low-connectivity user base. All interface text is in French (Locale fr_FR). The app uses a hybrid data strategy: a remote PHP API as the primary source, with a local SQLite database as an offline fallback seeded with default data on first install. All API calls are asynchronous with a 20-second timeout. Authentication and role enforcement are handled through SharedPreferences session persistence. The OTP code is currently hardcoded as "123456" for the prototype and is designed for replacement with an SMS gateway in production.

CHAPTER 3 — SYSTEM ARCHITECTURE

3.1 Layered Architecture

The application follows a four-layer architecture: a core layer (models, services, local database, constants), a feature layer (screens and providers per domain), a shared layer (reusable widgets and theming), and the application entry point. The feature layer is organised into domains — auth, booking, admin, home, tickets, search, map, notifications, and support — each containing its own screens and provider subdirectories, minimising coupling between features.

3.2 State Management and Data Flow

State management uses the Provider package (v6.1.1) with ChangeNotifier. Three providers are registered at the application root: AuthProvider (session and user state), BookingProvider (the multi-step booking flow), and AdminProvider (all administrative data). When a user triggers an action, the screen calls a provider method, which calls ApiService for a remote HTTP request. On success, the provider calls notifyListeners() and dependent widgets rebuild. When the remote API is unreachable, DatabaseHelper provides equivalent SQLite-backed methods.

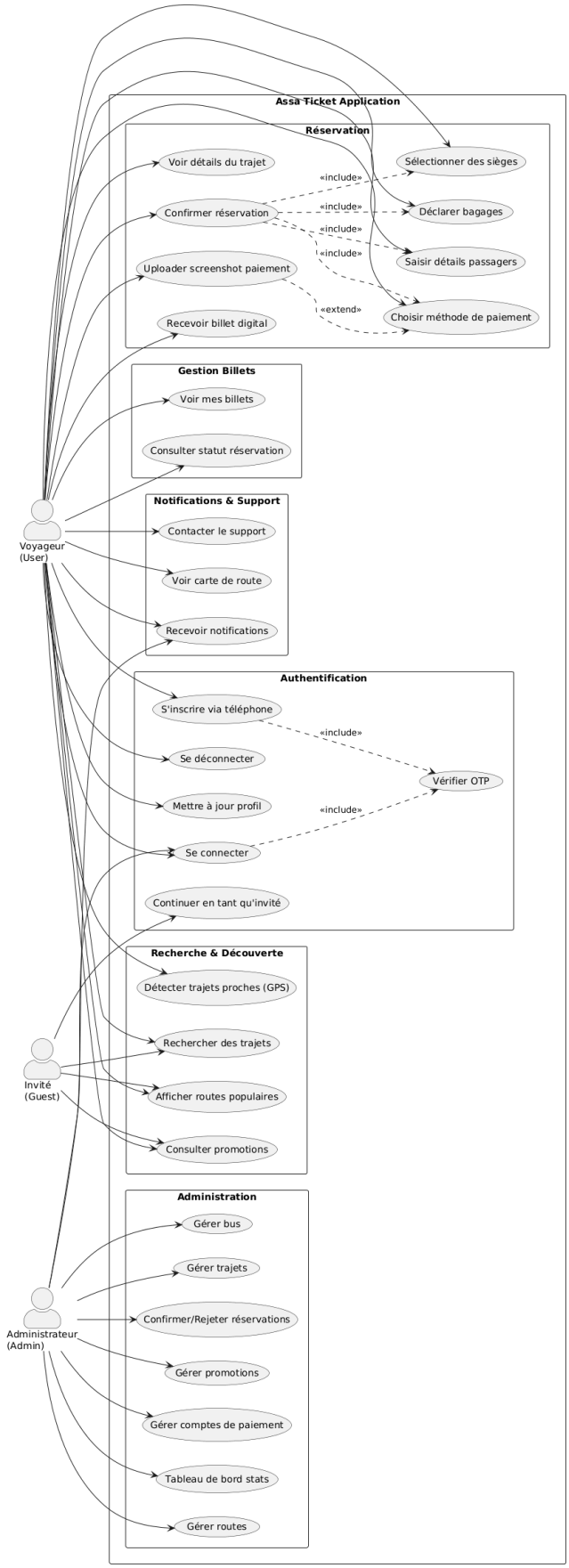
3.3 Navigation

Navigation is managed by go_router (v13.0.0) with 23 named routes declared centrally in AppRoutes. Routes are grouped by domain: authentication, user booking flow, and administration. This declarative model supports future deep-linking and web deployment without architectural changes.

CHAPTER 4 — UML DIAGRAMS

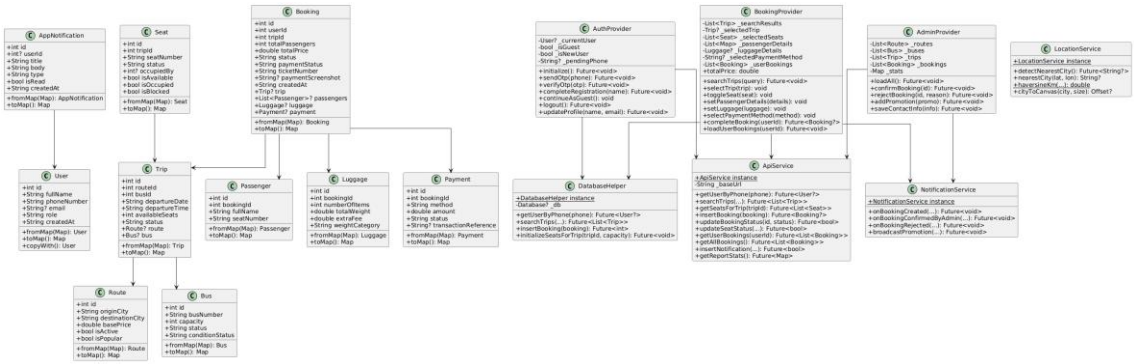
4.1 Use Case Diagram

The diagram below shows interactions between the three actors (Traveller, Guest, Administrator) and the system's functional use cases across six packages.


Assa Ticket Application
Réservation
Voir détails du trajet
Confirmer réservation
Uploader screenshot paiement
Recevoir billet digital
Sélectionner des sièges
Déclarer bagages
Saisir détails passagers
Choisir méthode de paiement
«include»
«include»
«include»
«include»
«extend»
Gestion Billets
Voir mes billets
Consulter statut réservation
Notifications & Support
Contacter le support
Voir carte de route
Recevoir notifications
Authentification
S'inscrire via téléphone
Se déconnecter
Mettre à jour profil
Se connecter
Continuer en tant qu'invité
Vérifier OTP
«include»
«include»
Recherche & Découverte
Détecter trajets proches (GPS)
Rechercher des trajets
Afficher routes populaires
Consulter promotions
Administration
Gérer bus
Gérer trajets
Confirmer/Rejeter réservations
Gérer promotions
Gérer comptes de paiement
Tableau de bord stats
Gérer routes
Voyageur
(User)
Invité
(Guest)
Administrateur
(Admin)

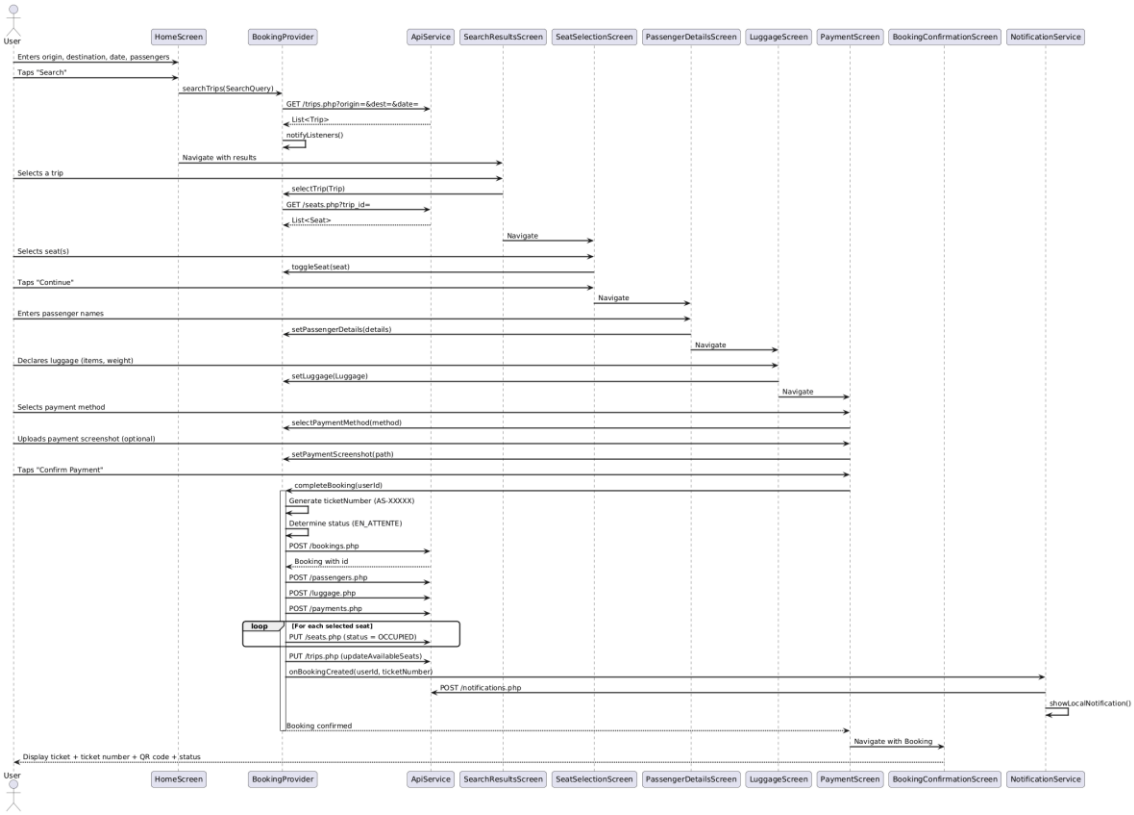
4.2 Class Diagram

The diagram below shows the core data models, service singletons, and provider classes with their key attributes, methods, and relationships.



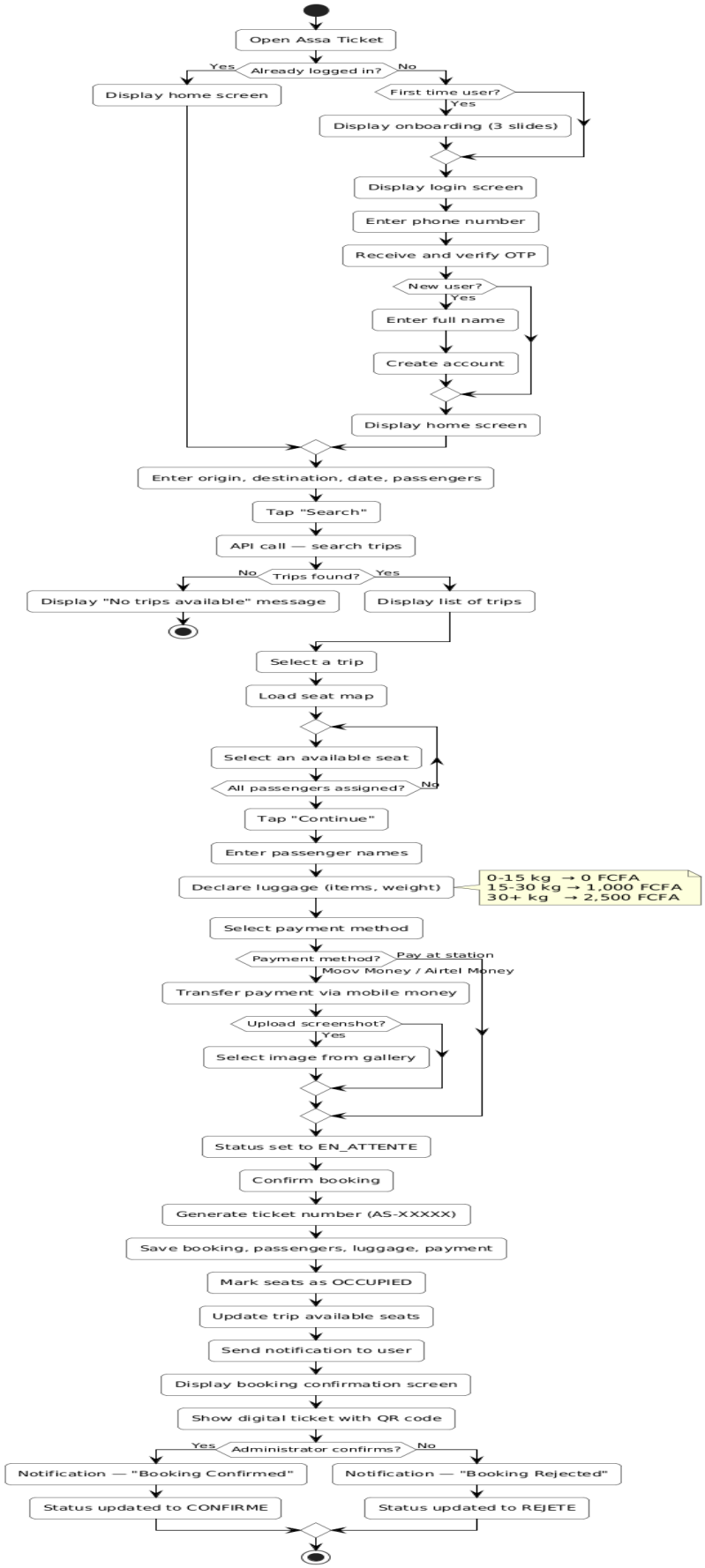
4.3 Sequence Diagram — Booking Process

The diagram below traces the booking flow from home screen search to confirmation, across screens, providers, the API service, and the notification service.



4.4 Activity Diagram — User Booking Flow

The diagram below models the full user journey from launch to booking confirmation, including all authentication, payment, and admin-confirmation decision branches.


Open Assa Ticket
Already logged in?
Yes
No
Display home screen
First time user?
Yes
Display onboarding (3 slides)
Display login screen
Enter phone number
Receive and verify OTP
New user?
Yes
Enter full name
Create account
Display home screen
Enter origin, destination, date, passengers
Tap "Search"
API call — search trips
Trips found?
No
Yes
Display "No trips available" message
Display list of trips
Select a trip
Load seat map
Select an available seat
All passengers assigned?
No
Tap "Continue"
Enter passenger names
Declare luggage (items, weight)
0-15 kg → 0 FCFA
15-30 kg → 1,000 FCFA
30+ kg → 2,500 FCFA
Select payment method
Payment method?
Pay at station
Moov Money / Airtel Money
Transfer payment via mobile money
Upload screenshot?
Yes
Select image from gallery
Status set to EN_ATTENTE
Confirm booking
Generate ticket number (AS-XXXXX)
Save booking, passengers, luggage, payment
Mark seats as OCCUPIED
Update trip available seats
Send notification to user
Display booking confirmation screen
Show digital ticket with QR code
Administrator confirms?
Yes
No
Notification — "Booking Confirmed"
Notification — "Booking Rejected"
Status updated to CONFIRME
Status updated to REJETE

CHAPTER 5 — DATABASE DESIGN

5.1 Overview

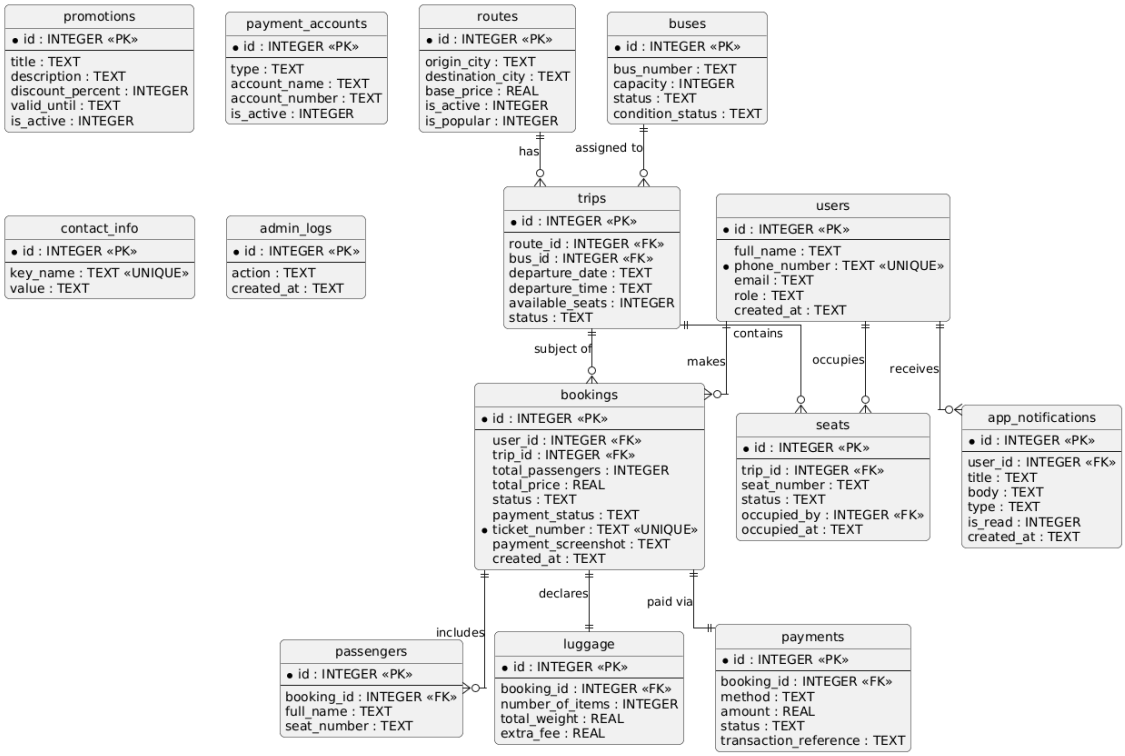
The application uses a dual-database strategy. A remote relational database accessed via a PHP REST API serves as the primary store. A local SQLite database (assa_ticket.db, schema version 2) provides an offline mirror seeded with default data on first installation. The schema comprises 14 tables across four domains: user and authentication, transport operations, commercial transactions, and system configuration.

5.2 Core Tables

Domain	Tables	Key Relationships
User & Auth	users	phone_number is UNIQUE; role is USER or ADMIN
Transport Operations	routes, buses, trips, seats	trips FK to routes and buses; seats FK to trips, one row per seat
Transactions	bookings, passengers, luggage, payments	bookings FK to users and trips; passengers, luggage, payments FK to bookings
Configuration	promotions, contact_info, app_notifications, admin_logs, payment_accounts	app_notifications FK to users; contact_info is a key-value store

5.3 Entity-Relationship Diagram

The ER diagram below illustrates all 14 tables and their relationships.


promotions
• id : INTEGER «PK»
title : TEXT
description : TEXT
discount_percent : INTEGER
valid_until : TEXT
is_active : INTEGER
payment_accounts
• id : INTEGER «PK»
type : TEXT
account_name : TEXT
account_number : TEXT
is_active : INTEGER
routes
• id : INTEGER «PK»
origin_city : TEXT
destination_city : TEXT
base_price : REAL
is_active : INTEGER
is_popular : INTEGER
buses
• id : INTEGER «PK»
bus_number : TEXT
capacity : INTEGER
status : TEXT
condition_status : TEXT
has
assigned to
trips
• id : INTEGER «PK»
route_id : INTEGER «FK»
bus_id : INTEGER «FK»
departure_date : TEXT
departure_time : TEXT
available_seats : INTEGER
status : TEXT
contact_info
• id : INTEGER «PK»
key_name : TEXT «UNIQUE»
value : TEXT
admin_logs
• id : INTEGER «PK»
action : TEXT
created_at : TEXT
users
• id : INTEGER «PK»
full_name : TEXT
• phone_number : TEXT «UNIQUE»
email : TEXT
role : TEXT
created_at : TEXT
contains
subject of
makes
occupies
receives
bookings
• id : INTEGER «PK»
user_id : INTEGER «FK»
trip_id : INTEGER «FK»
total_passengers : INTEGER
total_price : REAL
status : TEXT
payment_status : TEXT
• ticket_number : TEXT «UNIQUE»
payment_screenshot : TEXT
created_at : TEXT
seats
• id : INTEGER «PK»
trip_id : INTEGER «FK»
seat_number : TEXT
status : TEXT
occupied_by : INTEGER «FK»
occupied_at : TEXT
app_notifications
• id : INTEGER «PK»
user_id : INTEGER «FK»
title : TEXT
body : TEXT
type : TEXT
is_read : INTEGER
created_at : TEXT
declares
paid via
includes
passengers
• id : INTEGER «PK»
booking_id : INTEGER «FK»
full_name : TEXT
seat_number : TEXT
luggage
• id : INTEGER «PK»
booking_id : INTEGER «FK»
number_of_items : INTEGER
total_weight : REAL
extra_fee : REAL
payments
• id : INTEGER «PK»
booking_id : INTEGER «FK»
method : TEXT
amount : REAL
status : TEXT
transaction_reference : TEXT

CHAPTER 6 — IMPLEMENTATION

6.1 Technology Stack

Technology	Role
Flutter (SDK ≥ 3.0)	Cross-platform UI framework targeting Android and iOS from a single Dart codebase.
Dart	Application language; async/await model used throughout for non-blocking API and database calls.
sqflite 2.3.0	Local SQLite database via the DatabaseHelper singleton, with sqflite_common_ffi for desktop testing.
PHP REST API	Remote backend at http://169.239.251.102:280/~deubaybe.dounia/api; JSON over HTTP.
provider 6.1.1	Reactive state management (ChangeNotifier + MultiProvider).
go_router 13.0.0	Declarative navigation with 23 named routes.
qr_flutter 4.1.0	QR code generation on the digital ticket confirmation screen.
geolocator 11.0.0	GPS coordinates for nearest-city detection via the Haversine formula.
flutter_local_notifications 17.2.1	Local push notifications for booking events and promotions.

6.2 Key Implementation Highlights

Booking Completion

The completeBooking() method in BookingProvider executes a sequential transaction: generating a unique ticket number (AS-XXXXX), determining booking status based on payment method and screenshot presence, inserting the booking, passengers, luggage, and payment records via the API, updating seat statuses to OCCUPIED, recalculating trip availability, and dispatching a notification through NotificationService.

Seat Inventory

Seats are initialised at trip creation (one record per seat, status = AVAILABLE). During selection, toggleSeat() manages the local selection state. On booking completion, each selected seat is updated to OCCUPIED via the API, and available_seats on the trip record is recomputed directly from the seats table to ensure accuracy.

Dual-Channel Notifications

Every notification event (booking created, confirmed, rejected, or promotion broadcast) both writes a record to the app_notifications table and fires a local device notification via flutter_local_notifications. This ensures the user sees an immediate alert and can retrieve past notifications from the in-app notification centre at any time.

CHAPTER 7 — CONCLUSION

7.1 Summary

Assa Ticket addresses the core deficiencies of Chad's manual bus ticketing system by providing a fully digital, mobile-first booking platform. It eliminates the need for physical terminal visits, provides real-time seat visibility, generates tamper-evident digital tickets with QR codes, and integrates with the mobile money infrastructure already used by millions of Chadians. The French-language interface and simplified booking flow ensure accessibility for users across a range of digital literacy levels, while the administrator module gives bus operators full operational control within the same application.

7.2 Future Improvements

Key enhancements planned for a production release include: integration with a real SMS gateway (e.g., Africa's Talking) to replace the simulated OTP; direct API integration with Moov Money and Airtel Money to automate payment confirmation; WhatsApp ticket delivery via the WhatsApp Business API; an offline-first synchronisation mechanism for users in low-connectivity areas; real-time seat availability updates via WebSocket to prevent booking conflicts; Arabic language support for northern and eastern Chad; and expanded analytics in the admin dashboard.

Declaration on the Use of Artificial Intelligence

I, hereby declare that artificial intelligence tools were used during the preparation of this project and its accompanying report

Tools used: Claude (Anthropic) accessed via Claude Code CLI

How AI was used:

AI assistance was employed in the following specific ways:

- **Code assistance:** The AI tool was used during the development process to suggest implementation approaches, debug logic errors, and review Dart code structure. All final code decisions, architectural choices, and feature implementations were made, reviewed, and validated by me.
- **Repor t& ReadMe:** The academic report was generated by the AI tool based on a deep analysis of the actual project source files — including all Dart files, the database schema, the API service, models, providers, and screens. The AI was instructed to derive all descriptions strictly from the real codebase, with no invented features. All content in the report reflects the actual implemented system.
- **Presentation planning:** The AI was used to structure the video presentation outline, segment timing, and presenter script.

APPENDIX — BOOKING STATUS STATE MACHINE

Status	Meaning	Transitions To
EN_ATTENTE	Pending — initial state for all new bookings.	CONFIRME or REJETÉ
CONFIRME	Confirmed — administrator has approved the booking.	COMPLÉTÉ (after departure)
REJETÉ	Rejected — administrator has declined the booking.	Terminal state
COMPLÉTÉ	Completed — the trip has departed.	Terminal state
ANNULÉ	Cancelled — by user or administrator at any point.	Terminal state